

1. Visión General

Food Store es un sistema de comercio electrónico diseñado específicamente para la venta de productos alimenticios. Su propósito fundamental es ofrecer una plataforma completa que permita a los clientes explorar un catálogo de productos, gestionar un carrito de compras, realizar pedidos y pagar de forma segura a través de MercadoPago. Al mismo tiempo, el sistema brinda herramientas de administración para controlar el inventario, procesar pedidos y obtener métricas del negocio.

El sistema contempla cinco actores principales que interactúan con la plataforma de maneras distintas y complementarias:

- ****Cliente****: Es el usuario final de la tienda. Puede registrarse, iniciar sesión, navegar el catálogo de productos, agregar ítems a su carrito, crear pedidos, seleccionar una dirección de entrega, realizar pagos mediante MercadoPago y consultar el historial de sus pedidos. El cliente tiene visibilidad únicamente sobre sus propios datos y operaciones.

- ****Administrador (Admin)****: Posee control total sobre el sistema. Puede gestionar usuarios (crear, editar, desactivar), asignar roles, administrar el catálogo completo de productos y categorías, supervisar todos los pedidos sin importar su estado, acceder al panel de métricas y configurar parámetros globales del sistema como formas de pago y estados de pedido.

- ****Gestor de Stock****: Es el responsable de mantener actualizado el inventario. Puede consultar y modificar las cantidades disponibles de cada producto, marcar productos como disponibles o no disponibles, y gestionar los ingredientes asociados a cada producto incluyendo la identificación de alérgenos. Su alcance está limitado exclusivamente a las operaciones de catálogo e inventario.

- ****Gestor de Pedidos****: Se encarga del flujo operativo de los pedidos. Puede visualizar todos los pedidos del sistema, avanzar su estado a través de la máquina de estados definida (por ejemplo, de CONFIRMADO a EN_PREPARACIÓN, o de EN_PREPARACIÓN a EN_CAMINO), y cancelar pedidos cuando las reglas de negocio lo permitan. No tiene acceso a la gestión de productos ni de usuarios.

- ****Sistema****: Representa los procesos automatizados que operan sin intervención humana. El actor Sistema se encarga de recibir y procesar las notificaciones IPN (Instant Payment Notification) de MercadoPago, actualizar el estado de los pagos en la base de datos, y disparar

las transiciones automáticas de estado en los pedidos cuando un pago es aprobado. También gestiona la expiración y rotación de tokens de autenticación.

Los objetivos principales del sistema son: proporcionar una experiencia de compra fluida y segura para el cliente, garantizar la trazabilidad completa de cada pedido desde su creación hasta su entrega, mantener la integridad del inventario en todo momento, integrar de forma robusta el procesamiento de pagos con MercadoPago, y ofrecer un modelo de autorización granular basado en roles que permita segregar responsabilidades operativas.

2. Stack Tecnológico

Frontend

El frontend de Food Store está construido sobre **React** con **TypeScript**, utilizando **Vite** como bundler y servidor de desarrollo. La elección de Vite responde a su velocidad superior en comparación con alternativas como Webpack o Create React App, gracias a su uso de ES modules nativos durante el desarrollo y Rollup para las builds de producción.

Para la gestión del estado del servidor — es decir, los datos que provienen del backend como productos, pedidos y usuarios — se utiliza **TanStack Query** (anteriormente conocido como React Query). Esta librería maneja automáticamente el caching, la revalidación, el refetching en segundo plano, la paginación y los estados de carga y error. Esto permite que los componentes se suscriban a queries declarativas sin necesidad de escribir lógica imperativa de fetching.

La gestión de formularios se delega a **TanStack Form**, que proporciona validación declarativa, manejo de estado de campos individuales y soporte para esquemas de validación complejos. Esta librería se integra naturalmente con el ecosistema TanStack y permite construir formularios tipados de extremo a extremo gracias a TypeScript.

Para el estado puramente del cliente — como el carrito de compras, el estado de autenticación y preferencias de la interfaz — se emplea **Zustand**. Esta librería destaca por su API minimalista, su excelente rendimiento (ya que permite suscripciones granulares a porciones específicas del store) y su capacidad de persistir estado en localStorage mediante middleware.

Las peticiones HTTP al backend se realizan a través de **Axios**, configurado con interceptores que automáticamente adjuntan el token JWT a cada request y manejan la renovación transparente del token cuando éste expira.

La visualización de datos y métricas del panel de administración utiliza **recharts**, una librería de gráficos construida sobre componentes React y D3. Permite renderizar gráficos de barras, líneas, tortas y áreas de forma declarativa.

El sistema de estilos está basado en **Tailwind CSS**, un framework de utilidades que permite construir interfaces directamente en el markup sin necesidad de escribir hojas de estilo separadas. Tailwind se integra con Vite a través de PostCSS y ofrece purging automático de clases no utilizadas en producción.

Para la integración con MercadoPago en el lado del cliente, se utiliza el **SDK de MercadoPago para JavaScript** (MercadoPago.js), que permite la tokenización segura de tarjetas directamente en el navegador sin que los datos sensibles pasen por el servidor de Food Store, cumpliendo así con los requisitos de PCI DSS SAQ-A.

Backend

El backend está construido con **FastAPI**, un framework moderno de Python que ofrece alto rendimiento gracias a su naturaleza asíncrona basada en ASGI, validación automática de datos mediante Pydantic, y generación automática de documentación OpenAPI (Swagger UI y ReDoc).

El ORM elegido es **SQLModel**, una librería creada por el mismo autor de FastAPI (Sebastián Ramírez) que combina la potencia de SQLAlchemy con la validación de Pydantic en un solo modelo. Esto significa que los modelos de base de datos y los schemas de validación comparten una base común, reduciendo la duplicación de código y los errores de sincronización.

La base de datos es **PostgreSQL**, seleccionada por su robustez, soporte avanzado de tipos de datos (como arrays de enteros para la personalización de pedidos), soporte para Common Table Expressions (CTE) recursivas necesarias para las categorías jerárquicas, y su ecosistema maduro de herramientas.

Las migraciones de base de datos se gestionan con **Alembic**, la herramienta estándar del ecosistema SQLAlchemy. Alembic permite generar migraciones automáticas a partir de cambios

en los modelos, aplicarlas de forma incremental, y revertirlas cuando sea necesario. Cada migración se versiona como un script Python que describe los cambios DDL.

El hashing de contraseñas se realiza mediante **Passlib** con el algoritmo **bcrypt**. Passlib proporciona una interfaz unificada para múltiples algoritmos de hashing y maneja automáticamente la generación de salts, la verificación de hashes y la migración transparente entre algoritmos.

El rate limiting se implementa con **slowapi**, una librería que integra el rate limiting directamente en FastAPI a través de middleware. Se utiliza principalmente para proteger el endpoint de login contra ataques de fuerza bruta, limitando los intentos a 5 cada 15 minutos por dirección IP.

La generación y verificación de tokens JWT se realiza con **python-jose** (o **PyJWT** según la configuración), que proporciona las funciones criptográficas necesarias para firmar y validar tokens con el algoritmo HS256.

Para la integración con MercadoPago en el backend, se utiliza el **SDK oficial de MercadoPago para Python**, que proporciona clientes tipados para la API de Orders (checkout), gestión de pagos y procesamiento de webhooks.

3. Arquitectura del Sistema

Arquitectura del Backend

El backend de Food Store sigue una arquitectura en capas con flujo de dependencia unidireccional. Esto significa que cada capa solo conoce y depende de la capa inmediatamente inferior, nunca de las superiores. Esta restricción es fundamental para mantener la testabilidad, la mantenibilidad y la separación de responsabilidades.

Las capas, de exterior a interior, son las siguientes:

Router es la capa más externa y se encarga exclusivamente de recibir las peticiones HTTP, validar los datos de entrada mediante schemas Pydantic, invocar al servicio correspondiente, y devolver la respuesta HTTP apropiada. Los routers no contienen lógica de negocio — son

delegadores puros. Cada router está asociado a un módulo funcional (por ejemplo, ``router_productos.py``, ``router_pedidos.py``) y se registra en la aplicación FastAPI con un prefijo de ruta específico.

****Service**** es la capa que contiene toda la lógica de negocio. Aquí se implementan las reglas, validaciones, cálculos y orquestación de operaciones. Un servicio puede coordinar múltiples repositorios a través del Unit of Work, aplicar reglas de negocio como la validación de transiciones de estado, calcular totales de pedidos con sus snapshots, y decidir qué hacer en caso de error. Los servicios son clases o funciones que reciben el Unit of Work como dependencia.

****Unit of Work (UoW)**** es la capa que gestiona la transacción de base de datos. Encapsula una sesión de SQLAlchemy y expone los repositorios necesarios como atributos. El UoW se encarga de abrir la transacción, coordinar los commits y, en caso de error, ejecutar el rollback. Esto garantiza que operaciones complejas que involucran múltiples tablas (como crear un pedido con sus detalles, actualizar stock y registrar el historial de estados) sean atómicas.

****Repository**** es la capa de acceso a datos. Cada repositorio implementa operaciones CRUD sobre una entidad específica, abstrayendo los detalles de SQLAlchemy y SQLModel. Existe un ``BaseRepository[T]`` genérico que proporciona las operaciones comunes, y repositorios especializados que agregan queries específicas del dominio (como buscar productos por categoría o listar pedidos por estado).

****Model**** es la capa más interna y define las entidades de base de datos mediante clases SQLModel. Cada modelo mapea directamente a una tabla de PostgreSQL y define sus columnas, tipos, relaciones y restricciones.

El flujo de una petición típica es: el cliente envía un request HTTP → el Router lo recibe y valida los datos de entrada → invoca al Service pasándole el UoW → el Service ejecuta la lógica de negocio usando los Repositories del UoW → los Repositories interactúan con los Models para leer o escribir en la base de datos → el resultado asciende de vuelta por las capas hasta convertirse en una respuesta HTTP.

La organización del código sigue un enfoque ****feature-first**** (también llamado modular o vertical). En lugar de agrupar todos los routers en una carpeta, todos los servicios en otra y todos los modelos en otra, cada funcionalidad del sistema tiene su propia carpeta que contiene todos los archivos que necesita. Los módulos principales son:

- ****auth****: Login, registro, renovación de tokens, logout.
- ****refreshtokens****: Gestión del ciclo de vida de refresh tokens.

- **usuarios**: CRUD de usuarios, asignación de roles.
- **direcciones**: Gestión de direcciones de entrega del cliente.
- **categorías**: CRUD de categorías con soporte jerárquico.
- **productos**: CRUD de productos, ingredientes, relaciones con categorías.
- **pedidos**: Creación de pedidos, avance de estados, historial.
- **pagos**: Integración con MercadoPago, procesamiento de webhooks.
- **admin**: Panel de métricas y operaciones administrativas.

Cada módulo contiene típicamente: un archivo de modelo (`model.py`), un archivo de schemas Pydantic (`schemas.py`), un repositorio (`repository.py`), un servicio (`service.py`) y un router (`router.py`). Esta estructura hace que sea extremadamente claro dónde buscar y dónde agregar código relacionado con una funcionalidad específica.

Arquitectura del Frontend

El frontend sigue el patrón **Feature-Sliced Design (FSD)**, una metodología arquitectónica que organiza el código en capas horizontales y segmentos verticales. Las capas, de mayor a menor nivel de abstracción, son: app, pages, widgets, features, entities y shared. Cada capa solo puede importar de capas inferiores, nunca de superiores ni de la misma capa.

La capa **shared** contiene utilidades, componentes UI genéricos (botones, inputs, modals), configuración de Axios, tipos globales y constantes. La capa **entities** define los modelos de dominio del frontend y sus operaciones básicas (por ejemplo, la entidad Producto con su tipo TypeScript y su API de fetching). La capa **features** implementa interacciones de usuario específicas como "agregar al carrito", "realizar pago" o "filtrar productos". La capa **widgets** compone múltiples features y entities en bloques de interfaz más grandes. La capa **pages** define las páginas de la aplicación, cada una correspondiente a una ruta. La capa **app** configura providers, routing y estilos globales.

Una decisión arquitectónica fundamental en el frontend es la separación estricta entre estado del cliente y estado del servidor. **Zustand** se utiliza exclusivamente para estado que vive en el cliente: el carrito de compras (que se persiste en localStorage), el estado de autenticación (tokens JWT), el estado del proceso de pago, y preferencias de la interfaz como el tema visual o el estado de sidebars. **TanStack Query** se utiliza exclusivamente para datos que provienen del servidor: listados de productos, detalles de pedidos, información de usuarios, categorías, y cualquier dato que tenga una fuente de verdad en el backend. Esta separación evita los problemas clásicos de duplicación y desincronización de datos que surgen cuando se intenta manejar todo en un solo store global.

4. Modelo de Datos (ERD v5)

El modelo de datos de Food Store está organizado en tres dominios claramente diferenciados, todos diseñados en tercera forma normal (3NF) para garantizar la integridad referencial y minimizar la redundancia. A lo largo del modelo se aplican dos patrones transversales: el **soft delete** (borrado lógico mediante un campo `eliminado_en` de tipo timestamp nullable, que permite "eliminar" registros sin perderlos físicamente) y los **campos de auditoría** (`creado_en` y `actualizado_en` presentes en todas las tablas principales).

Dominio 1 — Identidad y Acceso

Este dominio gestiona todo lo relacionado con la autenticación, autorización y datos personales de los usuarios.

La entidad central es **Usuario**, que almacena los datos de cada persona registrada en el sistema. Sus campos principales son: un identificador único autoincremental, el nombre completo, el email (que sirve como credencial de login y tiene una restricción de unicidad), el hash de la contraseña (nunca se almacena la contraseña en texto plano), un teléfono opcional, y los campos de auditoría y soft delete. El email se indexa para optimizar las búsquedas durante el login.

La entidad **Rol** define los perfiles de autorización disponibles en el sistema. Es una tabla catálogo que se carga mediante seed data y contiene cuatro registros fijos: ADMIN, STOCK, PEDIDOS y CLIENT. Cada rol tiene un identificador, un nombre único y una descripción.

La relación entre usuarios y roles se modela mediante la tabla intermedia **UsuarioRol**, que implementa una relación muchos-a-muchos. Un usuario puede tener múltiples roles simultáneamente (por ejemplo, un usuario podría ser tanto ADMIN como STOCK), y un rol puede estar asignado a múltiples usuarios. Esta tabla contiene el ID del usuario, el ID del rol, y una restricción de unicidad compuesta que impide asignar el mismo rol dos veces al mismo usuario.

La entidad **RefreshToken** almacena los tokens de renovación emitidos durante el login. Cada registro contiene el token en sí (una cadena UUID única), el ID del usuario al que pertenece, la fecha de expiración, y un campo `revocado_en` que se establece cuando el token es invalidado (ya sea por logout explícito o por rotación). La relación con Usuario es

muchos-a-uno: un usuario puede tener múltiples refresh tokens activos (por ejemplo, desde diferentes dispositivos).

La entidad **DireccionEntrega** almacena las direcciones de envío de cada cliente. Un usuario puede tener múltiples direcciones registradas, y cada dirección incluye: calle, número, piso y departamento (opcionales), ciudad, código postal, y una referencia opcional para facilitar la entrega. También incluye un campo booleano `es_predeterminada` que indica cuál es la dirección por defecto del usuario. La relación con Usuario es muchos-a-uno.

Dominio 2 — Catálogo de Productos

Este dominio abarca la gestión del catálogo, incluyendo categorías, productos, ingredientes y sus relaciones.

La entidad **Categoria** implementa un sistema jerárquico de categorías. Cada categoría tiene un identificador, un nombre, una descripción, una imagen opcional, y crucialmente un campo `padre_id` que es una clave foránea autoreferencial apuntando a otra categoría. Este diseño permite construir árboles de categorías de profundidad arbitraria — por ejemplo, "Alimentos" → "Lácteos" → "Quesos" → "Quesos duros". Para consultar el árbol completo de una categoría (incluyendo todos sus descendientes), se utilizan **Common Table Expressions (CTE) recursivas** de PostgreSQL, lo que permite obtener toda la jerarquía en una sola query eficiente sin necesidad de múltiples round-trips a la base de datos.

La entidad **Producto** es el corazón del catálogo. Cada producto tiene: un identificador, un nombre, una descripción detallada, una URL de imagen, el precio unitario (almacenado como tipo numérico de precisión fija para evitar errores de punto flotante), la cantidad en stock (`stock_cantidad` como entero), un campo booleano `disponible` que permite desactivar un producto sin eliminarlo, y los campos de auditoría y soft delete. El campo `stock_cantidad` se decrementa atómicamente cuando se confirma un pedido y se incrementa si un pedido es cancelado, garantizando la consistencia del inventario.

La entidad **Ingrediente** registra los componentes de cada producto. Además de un nombre y una descripción, incluye el campo booleano `es_alergeno` que indica si el ingrediente es un alérgeno común (como gluten, lactosa, frutos secos, maní, etc.). Esta información es crítica para cumplir con regulaciones alimentarias y permitir que los clientes con restricciones dietarias tomen decisiones informadas.

Las relaciones entre productos y categorías se modelan mediante la tabla intermedia **ProductoCategoria**, que implementa una relación muchos-a-muchos. Un producto puede

pertenecer a múltiples categorías (por ejemplo, una pizza podría estar en "Comidas preparadas" y en "Ofertas"), y una categoría puede contener múltiples productos.

De manera análoga, la tabla **ProductoIngrediente** conecta productos con ingredientes en una relación muchos-a-muchos. Esto permite que un ingrediente como "Harina de trigo" esté asociado a múltiples productos, y que un producto tenga una lista completa de sus ingredientes.

La entidad **FormaPago** es una tabla catálogo que define los métodos de pago aceptados por la tienda. Se carga mediante seed data y contiene registros como "Tarjeta de crédito", "Tarjeta de débito", y potencialmente "Efectivo al recibir". Cada forma de pago tiene un identificador, un nombre y un booleano `activo` que permite habilitar o deshabilitar métodos de pago sin eliminarlos.

Dominio 3 — Ventas, Pagos y Trazabilidad

Este es el dominio más complejo y donde se aplican los patrones más sofisticados del sistema. Gestiona todo el ciclo de vida de un pedido desde su creación hasta su entrega, incluyendo el procesamiento de pagos.

La entidad **EstadoPedido** es una tabla catálogo que define los estados posibles de un pedido. Se carga mediante seed data y contiene seis registros: PENDIENTE, CONFIRMADO, EN_PREPARACIÓN, EN_CAMINO, ENTREGADO y CANCELADO. Cada estado tiene un identificador, un nombre único y una descripción.

La entidad **Pedido** es el agregado raíz de una orden de compra. Sus campos incluyen: el ID del usuario que realizó el pedido, el ID del estado actual, el ID de la dirección de entrega seleccionada, el ID de la forma de pago elegida, el costo de envío calculado, y crucialmente, **snapshots** de los datos que podrían cambiar en el futuro. Estos snapshots son campos que copian el valor al momento de la creación del pedido: `direccion\_snapshot` almacena una copia serializada de la dirección de entrega tal como era cuando se creó el pedido (porque el usuario podría modificar o eliminar la dirección original después), y campos similares para otros datos volátiles. El pedido también incluye un campo `total` que almacena el monto total calculado (suma de los subtotales de los detalles más el costo de envío). Los campos de auditoría y soft delete completan la entidad.

El patrón de **snapshot** es una decisión de diseño fundamental. En un sistema de e-commerce, los precios de los productos cambian, las direcciones se actualizan, y los datos se modifican constantemente. Sin snapshots, un pedido histórico podría mostrar información incorrecta — por ejemplo, un precio diferente al que el cliente efectivamente pagó. Los

snapshots garantizan que cada pedido preserve una fotografía inmutable de los datos relevantes al momento de su creación.

La entidad **DetallePedido** representa cada línea dentro de un pedido. Cada detalle tiene: el ID del pedido al que pertenece, el ID del producto, la cantidad solicitada, el precio unitario al momento del pedido (`precio_snapshot` — otro uso del patrón snapshot), el subtotal calculado ($\text{cantidad} \times \text{precio unitario}$), y un campo `personalizacion` de tipo `INTEGER[]` (array de enteros de PostgreSQL). Este campo de personalización almacena los IDs de los ingredientes que el cliente desea excluir del producto — por ejemplo, si pide una hamburguesa sin cebolla y sin lechuga, el array contendría los IDs de esos ingredientes. El uso de un array de enteros en lugar de una tabla intermedia es una decisión pragmática que simplifica las queries y el modelo sin sacrificar funcionalidad, dado que la personalización es un dato inmutable que se lee siempre como un todo.

La entidad **HistorialEstadoPedido** implementa un **audit trail append-only** (registro de auditoría de solo inserción). Cada vez que un pedido cambia de estado, se inserta un nuevo registro en esta tabla con: el ID del pedido, el ID del estado anterior, el ID del estado nuevo, el ID del usuario que ejecutó la transición (o null si fue el sistema automáticamente), una observación opcional, y un timestamp. Esta tabla NUNCA se actualiza ni se elimina — solo se insertan registros. Esto proporciona una trazabilidad completa e inmutable de todo el ciclo de vida de cada pedido, lo cual es invaluable para auditorías, resolución de disputas y análisis operativo.

La entidad **Pago** registra la información de cada transacción de pago asociada a un pedido. Sus campos incluyen: el ID del pedido, el monto pagado, el `mp_payment_id` (identificador del pago en MercadoPago), el `mp_status` (estado del pago según MercadoPago), el `external_reference` (referencia que Food Store envía a MercadoPago para identificar el pedido, típicamente el UUID del pedido), y el `idempotency_key` (clave única que garantiza que un mismo pago no se procese dos veces, incluso si MercadoPago envía múltiples webhooks para el mismo evento). Los campos de auditoría completan la entidad. La relación entre Pedido y Pago es uno-a-muchos: un pedido puede tener múltiples intentos de pago (por ejemplo, si el primer intento fue rechazado y el cliente reintenta).

5. Máquina de Estados del Pedido

El ciclo de vida de un pedido en Food Store se modela como una máquina de estados finitos (FSM) con seis estados y transiciones estrictamente controladas. Este diseño garantiza que un pedido solo pueda avanzar por caminos válidos y que cualquier intento de transición inválida sea rechazado por el sistema.

Los seis estados y sus significados son:

****PENDIENTE**** es el estado inicial de todo pedido recién creado. En este estado, el pedido ha sido registrado en el sistema pero aún no se ha recibido confirmación de pago. El pedido permanece en este estado hasta que MercadoPago notifica que el pago fue aprobado, o hasta que el cliente o un gestor lo cancela.

****CONFIRMADO**** es el estado al que transiciona un pedido cuando el pago asociado es aprobado por MercadoPago. En este momento, el sistema decrementa automáticamente el stock de cada producto incluido en el pedido. Este estado indica que el pedido es válido, pagado, y está listo para ser procesado por el equipo de preparación.

****EN_PREPARACIÓN**** indica que el equipo de cocina o preparación está trabajando activamente en el pedido. La transición a este estado es realizada por un usuario con rol PEDIDOS (Gestor de Pedidos) y marca el inicio del proceso operativo.

****EN_CAMINO**** indica que el pedido ha sido despachado y está en tránsito hacia la dirección de entrega del cliente. Esta transición también es realizada por el Gestor de Pedidos.

****ENTREGADO**** es un estado terminal que indica que el pedido fue recibido exitosamente por el cliente. Una vez en este estado, no se permiten más transiciones. Es el final feliz del flujo.

****CANCELADO**** es el otro estado terminal, pero representa la finalización no exitosa del pedido. Un pedido puede ser cancelado desde múltiples estados previos, pero no desde todos. Una vez cancelado, si el pedido ya había sido confirmado (es decir, el stock ya se había decrementado), el sistema debe restaurar el stock de los productos afectados.

Las transiciones válidas son:

- PENDIENTE → CONFIRMADO (automática, cuando el pago es aprobado)
- CONFIRMADO → EN_PREPARACIÓN (manual, por Gestor de Pedidos)
- EN_PREPARACIÓN → EN_CAMINO (manual, por Gestor de Pedidos)
- EN_CAMINO → ENTREGADO (manual, por Gestor de Pedidos)
- PENDIENTE → CANCELADO (manual, por Cliente, Gestor de Pedidos o Admin)

- CONFIRMADO → CANCELADO (manual, por Gestor de Pedidos o Admin, con restauración de stock)
- EN_PREPARACIÓN → CANCELADO (manual, por Admin únicamente, con restauración de stock)

Las reglas de negocio asociadas a la máquina de estados son:

****RN-01****: Un pedido solo puede avanzar al siguiente estado en la secuencia definida. No se permiten saltos (por ejemplo, de PENDIENTE directamente a EN_CAMINO) ni retrocesos (por ejemplo, de EN_PREPARACIÓN a CONFIRMADO). La única excepción es la transición a CANCELADO, que puede ocurrir desde estados no terminales.

****RN-02****: La transición de PENDIENTE a CONFIRMADO es exclusivamente automática y se dispara cuando el sistema recibe una notificación IPN de MercadoPago indicando que el pago fue aprobado (`mp_status = "approved"`). Ningún usuario puede ejecutar esta transición manualmente.

****RN-03****: Cuando un pedido alcanza el estado CONFIRMADO, el sistema debe decrementar atómicamente el `stock_cantidad` de cada producto incluido en el pedido. Esta operación debe ser transaccional — si el decremento de cualquier producto falla (por ejemplo, porque no hay stock suficiente), toda la operación debe revertirse y el pedido debe permanecer en PENDIENTE.

****RN-04****: Cuando un pedido que ya fue confirmado es cancelado, el sistema debe restaurar el stock de todos los productos afectados. Esta restauración es el proceso inverso de RN-03 y también debe ser atómica.

****RN-05****: Los estados ENTREGADO y CANCELADO son terminales. Una vez que un pedido alcanza cualquiera de estos estados, no se permite ninguna transición adicional. Cualquier intento de modificar el estado de un pedido terminal debe ser rechazado con un error descriptivo.

6. Autenticación y Autorización

Flujo de Autenticación JWT

Food Store implementa un sistema de autenticación basado en JSON Web Tokens (JWT) con un esquema de doble token: un **access token** de corta duración y un **refresh token** de larga duración.

El **access token** tiene una duración de 30 minutos y se utiliza para autenticar cada petición al backend. Su payload contiene el ID del usuario, su email y sus roles. Se firma con el algoritmo HS256 usando una clave secreta configurada en las variables de entorno. Este token se envía en el header `Authorization: Bearer <token>` de cada petición HTTP.

El **refresh token** tiene una duración de 7 días y se utiliza exclusivamente para obtener un nuevo access token cuando el actual expira. Se genera como un UUID v4 y se almacena en la base de datos en la tabla RefreshToken, asociado al usuario. A diferencia del access token, el refresh token no contiene información del usuario — es simplemente un identificador opaco que el servidor valida contra la base de datos.

El flujo completo de autenticación funciona de la siguiente manera:

Cuando el usuario inicia sesión, envía su email y contraseña al endpoint de login. El backend verifica las credenciales contra la base de datos (comparando el hash bcrypt), y si son válidas, genera ambos tokens. El access token se devuelve en el cuerpo de la respuesta junto con el refresh token y los datos básicos del usuario. El frontend almacena ambos tokens en el **authStore de Zustand**, que a su vez los persiste en localStorage mediante el middleware de persistencia.

Para cada petición subsiguiente, el **interceptor de Axios** automáticamente adjunta el access token al header Authorization. Cuando el access token expira (el servidor responde con 401), el interceptor automáticamente envía el refresh token al endpoint de renovación, obtiene un nuevo par de tokens, actualiza el authStore, y reintenta la petición original de forma transparente para el usuario. Este proceso ocurre sin que el usuario perciba ninguna interrupción.

El endpoint de renovación implementa **rotación de refresh tokens**: cada vez que se usa un refresh token para obtener uno nuevo, el token anterior se marca como revocado (se establece el campo `revocado_en`) y se emite uno completamente nuevo. Esto limita la ventana de exposición si un refresh token es comprometido.

El logout se implementa marcando el refresh token actual como revocado en la base de datos y limpiando el authStore en el frontend. No es necesario invalidar el access token explícitamente — dado su corta duración de 30 minutos, simplemente se deja expirar.

En el backend, la autenticación se implementa mediante la dependencia de FastAPI ``get_current_user``. Esta función se inyecta en cualquier endpoint que requiera autenticación, decodifica el access token del header Authorization, verifica su firma y expiración, y devuelve el objeto Usuario correspondiente. Si el token es inválido o está expirado, la dependencia lanza una excepción HTTP 401 automáticamente.

Control de Acceso Basado en Roles (RBAC)

La autorización en Food Store sigue el modelo RBAC (Role-Based Access Control) con cuatro roles predefinidos:

****ADMIN**** tiene acceso total al sistema. Puede gestionar usuarios y sus roles, administrar todo el catálogo de productos y categorías, supervisar y modificar todos los pedidos, acceder al panel de métricas, y configurar parámetros del sistema. Es el único rol que puede cancelar pedidos que ya están en preparación.

****STOCK**** (Gestor de Stock) tiene permisos limitados al catálogo. Puede crear, editar y desactivar productos, gestionar ingredientes y alérgenos, modificar cantidades de stock, y administrar categorías. No tiene acceso a pedidos, usuarios ni métricas.

****PEDIDOS**** (Gestor de Pedidos) tiene permisos limitados a la operación de pedidos. Puede ver todos los pedidos del sistema, avanzar el estado de los pedidos siguiendo la máquina de estados, y cancelar pedidos que estén en estado PENDIENTE o CONFIRMADO. No tiene acceso al catálogo ni a la gestión de usuarios.

****CLIENT**** es el rol asignado automáticamente a cada usuario que se registra. Puede ver el catálogo de productos, gestionar su carrito de compras, crear pedidos, realizar pagos, ver el historial de sus propios pedidos, y gestionar sus direcciones de entrega. Un cliente solo puede ver y operar sobre sus propios datos — nunca los de otros usuarios.

La verificación de roles se implementa mediante la dependencia ``require_role`` de FastAPI, que es un generador de dependencias parametrizado. Se utiliza de la siguiente manera: un endpoint que requiere rol ADMIN declarará ``require_role(["ADMIN"])` como dependencia, y uno que permite tanto ADMIN como PEDIDOS declarará ``require_role(["ADMIN", "PEDIDOS"])`. Si el usuario autenticado no posee ninguno de los roles requeridos, la dependencia lanza una excepción HTTP 403 (Forbidden).

Rate Limiting

Para proteger el sistema contra ataques de fuerza bruta, el endpoint de login implementa rate limiting mediante la librería slowapi. La configuración limita a **5 intentos de login cada 15 minutos** por dirección IP. Cuando se excede el límite, el servidor responde con HTTP 429 (Too Many Requests) e incluye un header `Retry-After` que indica cuántos segundos debe esperar el cliente antes de reintentar. Este mecanismo es transparente para usuarios legítimos (5 intentos en 15 minutos es más que suficiente para un humano) pero efectivo contra scripts automatizados.

7. API REST

Filosofía de Diseño

La API de Food Store sigue principios RESTful consistentes a lo largo de todos sus endpoints. Todas las rutas están prefijadas con `/api/v1` para permitir versionado futuro sin romper clientes existentes. Los errores se devuelven siguiendo el estándar **RFC 7807** (Problem Details for HTTP APIs), que define una estructura JSON consistente con campos como `type`, `title`, `status`, `detail` e `instance`. Esto permite que los clientes manejen errores de forma uniforme sin necesidad de interpretar formatos ad-hoc.

Los listados implementan **paginación** mediante parámetros `skip` (offset) y `limit` (cantidad), con valores por defecto razonables. Las respuestas paginadas incluyen metadatos como el total de registros disponibles para que el frontend pueda construir controles de paginación.

El filtrado por soft delete se maneja de forma transparente: por defecto, todos los endpoints de listado excluyen los registros con `eliminado\_en` no nulo. Los endpoints de administración pueden incluir un parámetro `incluir\_eliminados` para ver también los registros borrados lógicamente.

Módulo Auth

El módulo de autenticación expone los endpoints fundamentales para el ciclo de vida de la sesión del usuario.

El endpoint ****POST /api/v1/auth/login**** recibe un email y una contraseña, los valida contra la base de datos, y si son correctos devuelve un objeto con el access token, el refresh token, el tipo de token ("Bearer") y los datos del usuario incluyendo sus roles. Si las credenciales son inválidas, devuelve 401. Si se excede el rate limit, devuelve 429.

El endpoint ****POST /api/v1/auth/register**** permite la creación de nuevas cuentas de cliente. Recibe nombre, email, contraseña y teléfono opcional. Verifica que el email no esté registrado (devuelve 409 si ya existe), hashea la contraseña con bcrypt, crea el usuario, le asigna automáticamente el rol CLIENT, y devuelve los tokens y datos del usuario como si fuera un login automático post-registro.

El endpoint ****POST /api/v1/auth/refresh**** recibe un refresh token válido, verifica que exista en la base de datos, que no esté revocado, y que no haya expirado. Si todo es correcto, revoca el token actual, emite un nuevo par de tokens (access + refresh), y los devuelve. Si el refresh token es inválido, devuelve 401.

El endpoint ****POST /api/v1/auth/logout**** recibe el refresh token y lo marca como revocado en la base de datos. El access token simplemente se elimina del lado del cliente. Devuelve 204 (No Content) en caso de éxito.

Módulo Productos

El módulo de productos gestiona el catálogo completo de la tienda.

El endpoint ****GET /api/v1/productos**** devuelve el listado paginado de productos disponibles. Soporta filtros opcionales por categoría, por nombre (búsqueda parcial), por rango de precio, y por disponibilidad. Para el público general, solo devuelve productos con `disponible = true` y `eliminado\_en IS NULL`. Para usuarios con rol STOCK o ADMIN, puede incluir productos no disponibles.

El endpoint ****GET /api/v1/productos/{id}**** devuelve el detalle completo de un producto específico, incluyendo sus categorías asociadas, sus ingredientes (con la información de alérgenos), el stock disponible y todas las imágenes.

El endpoint ****POST /api/v1/productos**** permite crear un nuevo producto. Requiere rol STOCK o ADMIN. Recibe los datos del producto incluyendo precio, stock inicial, categorías e ingredientes. Valida que las categorías e ingredientes referenciados existan.

El endpoint **PUT /api/v1/productos/{id}** permite actualizar un producto existente. Requiere rol STOCK o ADMIN. Permite modificar cualquier campo incluyendo precio, stock, disponibilidad, categorías e ingredientes.

El endpoint **DELETE /api/v1/productos/{id}** realiza un soft delete del producto (establece ``eliminado_en``). Requiere rol STOCK o ADMIN. El producto no desaparece de la base de datos — simplemente deja de mostrarse en los listados públicos.

Módulo Pedidos

El módulo de pedidos es el más complejo de la API y gestiona todo el ciclo de vida de las órdenes.

El endpoint **POST /api/v1/pedidos** permite a un cliente crear un nuevo pedido. Recibe la lista de ítems (cada uno con ID de producto, cantidad y personalización opcional), el ID de la dirección de entrega y el ID de la forma de pago. El servicio valida que todos los productos existan y estén disponibles, verifica que haya stock suficiente para cada ítem, calcula los subtotales usando los precios actuales (creando snapshots), calcula el costo de envío, crea el pedido en estado PENDIENTE con todos sus detalles, y registra la entrada inicial en el historial de estados. Todo esto ocurre dentro de una única transacción.

El endpoint **GET /api/v1/pedidos** devuelve los pedidos según el rol del usuario. Si es CLIENT, devuelve solo sus propios pedidos. Si es PEDIDOS o ADMIN, devuelve todos los pedidos del sistema. Soporta filtros por estado, por fecha, y paginación.

El endpoint **GET /api/v1/pedidos/{id}** devuelve el detalle completo de un pedido incluyendo todos sus ítems con sus snapshots de precio, el historial de estados con timestamps y usuarios responsables, la dirección de entrega (snapshot), y la información de pagos asociados.

El endpoint **PATCH /api/v1/pedidos/{id}/avanzar** permite avanzar el estado de un pedido al siguiente estado válido según la máquina de estados. Recibe opcionalmente una observación que se registra en el historial. Valida que la transición sea válida según las reglas de negocio y que el usuario tenga el rol necesario para ejecutarla. En el caso de la transición a CONFIRMADO, también ejecuta el decremento de stock.

El endpoint **PATCH /api/v1/pedidos/{id}/cancelar** permite cancelar un pedido. Valida que el pedido esté en un estado cancelable y que el usuario tenga permiso para cancelarlo según su

rol. Si el pedido ya había sido confirmado, ejecuta la restauración de stock. Registra la transición en el historial con la observación proporcionada.

Módulo Pagos

El módulo de pagos gestiona la integración con MercadoPago.

El endpoint ****POST /api/v1/pagos/crear-preferencia**** (o su equivalente para la API de Orders) recibe el ID de un pedido en estado PENDIENTE y crea una preferencia de pago en MercadoPago. Esta preferencia incluye los ítems del pedido con sus precios snapshot, la URL de retorno para redireccionar al cliente después del pago, la URL del webhook para recibir notificaciones IPN, y el ``external_reference`` que vincula la preferencia con el pedido en Food Store. Devuelve el ID de la preferencia y la URL de pago para redirigir al cliente.

El endpoint ****POST /api/v1/pagos/webhook**** (o ****POST /api/v1/pagos/ipn****) recibe las notificaciones de MercadoPago. Cuando MercadoPago notifica un evento de pago, este endpoint extrae el ID del pago, consulta la API de MercadoPago para obtener el estado actual, actualiza el registro de pago en la base de datos, y si el estado es "approved", dispara automáticamente la transición del pedido de PENDIENTE a CONFIRMADO. Este endpoint debe responder con 200 rápidamente para que MercadoPago no reintente el webhook.

El endpoint ****GET /api/v1/pagos/pedido/{pedido_id}**** devuelve todos los pagos asociados a un pedido específico, permitiendo ver el historial de intentos de pago incluyendo los rechazados.

8. Schemas Pydantic v2

Food Store utiliza Pydantic v2 para la validación de datos de entrada y la serialización de respuestas. Los schemas siguen una convención estricta de separación en tres variantes por entidad:

- ****Create**** (o Request): Define los campos necesarios para crear un recurso. Solo incluye los campos que el cliente debe proporcionar. Ejemplo: ``RegisterRequest`` incluye nombre, email y contraseña, pero no incluye ID ni fechas de auditoría.

- ****Update****: Define los campos que pueden ser modificados. Generalmente todos los campos son opcionales (usando `Optional[T]`), permitiendo actualizaciones parciales donde el cliente solo envía los campos que desea cambiar.
- ****Read**** (o Response): Define la estructura de datos que se devuelve al cliente. Incluye el ID, los campos de la entidad, datos de relaciones anidadas cuando corresponde, y excluye campos sensibles como hashes de contraseña.

Schemas de Autenticación

****LoginRequest**** contiene dos campos obligatorios: ``email`` (validado como formato de email) y ``password`` (string con longitud mínima de 8 caracteres). Es el schema que recibe el endpoint de login.

****RegisterRequest**** extiende los campos de login con: ``nombre`` (string con longitud mínima de 2 caracteres), ``telefono`` (string opcional con validación de formato), y opcionalmente ``password_confirmation`` para validación en el frontend. El validator de Pydantic verifica que la contraseña cumple con requisitos mínimos de complejidad.

****TokenResponse**** es el schema de respuesta del login y el refresh. Contiene: ``access_token`` (string JWT), ``refresh_token`` (string UUID), ``token_type`` (siempre "Bearer"), y ``user`` (un objeto `UserResponse` anidado con los datos del usuario).

****UserResponse**** contiene los datos públicos del usuario: ``id``, ``nombre``, ``email``, ``telefono``, ``roles`` (lista de strings con los nombres de los roles), ``creado_en`` y ``actualizado_en``. Nunca incluye el hash de la contraseña.

Schemas de Pedidos

****CrearPedidoRequest**** es el schema de entrada para crear un pedido. Contiene: ``items`` (lista no vacía de `ItemPedidoRequest``), ``direccion_id`` (entero, ID de la dirección de entrega), y ``forma_pago_id`` (entero, ID de la forma de pago). Incluye un validator que verifica que la lista de ítems no esté vacía.

****ItemPedidoRequest**** define cada línea del pedido. Contiene: ``producto_id`` (entero), ``cantidad`` (entero positivo, mínimo 1), y ``personalizacion`` (lista opcional de enteros, representando los IDs de ingredientes a excluir). Si la personalización no se proporciona, se asume una lista vacía.

****AvanzarEstadoRequest**** es el schema para las transiciones de estado. Contiene un único campo opcional: ``observacion`` (string, máximo 500 caracteres) que se registra en el historial de estados para documentar el motivo de la transición.

****PedidoRead**** es el schema de respuesta para listados de pedidos. Contiene: ``id``, ``estado`` (nombre del estado actual), ``total``, ``costo_envio``, ``creado_en``, ``actualizado_en``, y datos resumidos del usuario y la dirección. No incluye los detalles de cada ítem para mantener las respuestas de listado livianas.

****PedidoDetail**** extiende `PedidoRead` con información completa. Agrega: ``detalles`` (lista de ``DetallePedidoRead`` con cada ítem), ``historial_estados`` (lista ordenada cronológicamente de todas las transiciones), ``pagos`` (lista de pagos asociados), y los snapshots completos de dirección y forma de pago.

****DetallePedidoRead**** contiene la información de cada línea del pedido: ``id``, ``producto_id``, ``producto_nombre``, ``cantidad``, ``precio_unitario`` (el snapshot del precio), ``subtotal``, y ``personalizacion`` (lista de IDs de ingredientes excluidos con sus nombres para facilitar la visualización).

9. Unit of Work

El Patrón

El Unit of Work (UoW) es un patrón de diseño que agrupa múltiples operaciones de base de datos en una única transacción lógica. En Food Store, el UoW encapsula una sesión de SQLAlchemy y expone todos los repositorios necesarios como atributos. Cuando el servicio comienza una operación, el UoW abre una transacción. Todas las operaciones de los repositorios se ejecutan dentro de esa transacción. Si todo sale bien, el UoW hace commit. Si algo falla, el UoW ejecuta rollback y ninguna de las operaciones se persiste. Esto garantiza la atomicidad — o todo se guarda o nada se guarda.

El UoW se implementa como un context manager de Python, permitiendo su uso con la sintaxis ``with`` (o ``async with`` en el caso asíncrono). Al entrar al contexto, se crea la sesión y se inicializan los repositorios. Al salir exitosamente, se hace commit. Si se lanza una excepción dentro del contexto, se hace rollback automáticamente.

Flujo de Creación de un Pedido

Para ilustrar el poder del patrón UoW, veamos el flujo completo de creación de un pedido, que es la operación más compleja del sistema e involucra múltiples tablas en una sola transacción:

****Paso 1 — Validación del usuario y la dirección**:** El servicio utiliza el repositorio de usuarios para verificar que el usuario existe y está activo, y el repositorio de direcciones para verificar que la dirección proporcionada pertenece a ese usuario.

****Paso 2 — Validación de la forma de pago**:** Se verifica que la forma de pago exista y esté activa mediante el repositorio de formas de pago.

****Paso 3 — Validación de productos y stock**:** Para cada ítem del pedido, el servicio utiliza el repositorio de productos para obtener el producto, verificar que esté disponible (``disponible = true``), que no haya sido eliminado (``eliminado_en IS NULL``), y que tenga stock suficiente (``stock_cantidad >= cantidad solicitada``). Si alguna validación falla, se lanza una excepción inmediatamente.

****Paso 4 — Creación de snapshots**:** Para cada producto, se captura el precio actual como ``precio_snapshot``. Para la dirección de entrega, se serializa la dirección completa como ``direccion_snapshot``. Estos snapshots garantizan que los datos del pedido sean inmutables independientemente de cambios futuros.

****Paso 5 — Cálculo de totales**:** Se calcula el subtotal de cada línea (`cantidad × precio snapshot`), el subtotal general (suma de todos los subtotales de línea), el costo de envío según las reglas de negocio, y el total final.

****Paso 6 — Creación del pedido**:** Se crea el registro en la tabla Pedido con el estado PENDIENTE, los totales calculados, y los snapshots. Se usa el repositorio de pedidos.

****Paso 7 — Creación de los detalles**:** Para cada ítem, se crea un registro en DetallePedido con el producto, cantidad, precio snapshot, subtotal y personalización. Se usa el repositorio de detalles de pedido.

****Paso 8 — Registro en historial**:** Se crea el registro inicial en HistorialEstadoPedido, documentando la creación del pedido (transición de null a PENDIENTE).

****Paso 9 — Commit****: Si todos los pasos anteriores se completaron sin error, el UoW ejecuta commit y todos los registros se persisten atómicamente.

****En caso de error**** en cualquier paso (por ejemplo, stock insuficiente en el paso 3, o un error de base de datos en el paso 7), el UoW ejecuta rollback automáticamente. Esto significa que no queda ningún registro parcial — ni el pedido incompleto, ni detalles huérfanos, ni historial sin pedido. El sistema mantiene su consistencia.

BaseRepository[T]

El `BaseRepository[T]` es una clase genérica que proporciona operaciones CRUD comunes para cualquier entidad. Se parametriza con el tipo de modelo `SQLModel(T)` y recibe la sesión de base de datos en su constructor. Sus métodos son:

- **`**get_by_id(id)**`**: Busca un registro por su clave primaria. Retorna el objeto o `None` si no existe. Por defecto, excluye registros con soft delete.
- **`**list_all(skip, limit, filters)**`**: Devuelve una lista paginada de registros. Acepta un offset (`skip`), un límite (`limit`), y filtros opcionales. Excluye registros eliminados por defecto.
- **`**count(filters)**`**: Devuelve el total de registros que coinciden con los filtros proporcionados, excluyendo eliminados. Útil para paginación.
- **`**create(obj)**`**: Recibe un objeto del tipo `T`, lo agrega a la sesión, ejecuta flush (para obtener el ID generado sin hacer commit), y retorna el objeto con su ID.
- **`**update(id, data)**`**: Busca el registro por ID, actualiza los campos proporcionados en `data`, ejecuta flush, y retorna el objeto actualizado. Si el registro no existe, lanza una excepción.
- **`**soft_delete(id)**`**: Busca el registro por ID y establece `eliminado_en` con el timestamp actual. No elimina el registro físicamente.
- **`**hard_delete(id)**`**: Elimina el registro físicamente de la base de datos. Se usa raramente — solo para datos que no necesitan preservarse como refresh tokens expirados.

Los repositorios especializados heredan de ``BaseRepository[T]`` y agregan métodos específicos del dominio. Por ejemplo, ``ProductoRepository`` agrega ``buscar_por_categoria(categoria_id)`` y ``actualizar_stock(producto_id, cantidad)``. ``PedidoRepository`` agrega ``listar_por_usuario(usuario_id)`` y ``listar_por_estado(estado_id)``.

10. Integración MercadoPago

Checkout API con Orders

La integración de pagos en Food Store utiliza la **API de Orders (Checkout)** de MercadoPago. Cuando un cliente está listo para pagar su pedido, el backend crea una orden en MercadoPago que contiene los ítems del pedido con sus precios snapshot, la información del pagador, y las URLs de callback. MercadoPago devuelve una URL de checkout a la que el cliente es redirigido para completar el pago de forma segura en el entorno de MercadoPago.

Cumplimiento PCI SAQ-A

Food Store cumple con el nivel **PCI DSS SAQ-A** de seguridad en el manejo de datos de tarjetas. Esto significa que los datos sensibles de las tarjetas de crédito (número, CVV, fecha de vencimiento) **NUNCA** pasan por el servidor de Food Store. La tokenización ocurre directamente en el navegador del cliente mediante el SDK de JavaScript de MercadoPago (`MercadoPago.js`). El SDK captura los datos de la tarjeta, los envía directamente a los servidores de MercadoPago, y devuelve un token que representa la tarjeta de forma segura. Este token es lo que Food Store envía a su backend para crear el pago — el backend nunca ve ni almacena datos reales de tarjetas.

Webhooks IPN

Las **Instant Payment Notifications (IPN)** son el mecanismo mediante el cual MercadoPago notifica a Food Store sobre cambios en el estado de los pagos. Cuando un pago cambia de estado (por ejemplo, de "pending" a "approved"), MercadoPago envía un POST al webhook configurado en Food Store.

El flujo del webhook es:

El endpoint recibe la notificación con el tipo de evento y el ID del recurso. Verifica que la notificación sea legítima (validación de firma o headers de MercadoPago). Consulta la API de MercadoPago para obtener el estado actual y completo del pago (nunca confía únicamente en los datos del webhook, siempre verifica). Busca el pedido correspondiente usando el `external\_reference`. Actualiza el registro de pago en la base de datos con el nuevo estado. Si corresponde, ejecuta acciones automáticas basadas en el estado. Responde con HTTP 200 inmediatamente para evitar reintentos.

Clave de Idempotencia

El campo `idempotency\_key` en la entidad Pago es una clave única que previene el procesamiento duplicado de pagos. MercadoPago puede enviar múltiples webhooks para el mismo evento (por ejemplo, si el primer intento no recibió un 200 a tiempo), y la clave de idempotencia garantiza que el sistema procese cada pago exactamente una vez. Antes de procesar un webhook, el servicio verifica si ya existe un registro con esa `idempotency\_key` — si existe, ignora la notificación duplicada.

Estados de Pago y Acciones del Sistema

Cada estado de pago en MercadoPago dispara una acción específica en Food Store:

****approved**** indica que el pago fue aprobado exitosamente. El sistema registra el pago con estado "approved", y automáticamente transiciona el pedido de PENDIENTE a CONFIRMADO, lo cual a su vez dispara el decremento de stock de todos los productos del pedido.

****pending**** indica que el pago está en proceso pero aún no fue confirmado (por ejemplo, un pago en efectivo que el cliente aún no realizó, o una transferencia bancaria pendiente). El sistema registra el pago con estado "pending" pero no modifica el estado del pedido — permanece en PENDIENTE.

****rejected**** indica que el pago fue rechazado (tarjeta sin fondos, datos incorrectos, fraude detectado). El sistema registra el pago con estado "rejected". El pedido permanece en PENDIENTE, permitiendo al cliente reintentar el pago con un método diferente.

****in_process**** indica que MercadoPago está revisando el pago (verificación antifraude en curso). Similar a "pending", el sistema registra el estado pero no modifica el pedido.

****cancelled**** indica que el pago fue cancelado, ya sea por el cliente o por MercadoPago. El sistema registra el pago con estado "cancelled". Si no hay otros pagos pendientes o aprobados para ese pedido, se podría considerar la cancelación automática del pedido según las reglas de negocio configuradas.

Tarjetas de Prueba en Sandbox

Para el desarrollo y testing, MercadoPago proporciona un entorno Sandbox con tarjetas de prueba que simulan diferentes resultados:

- Tarjetas que siempre resultan en pago aprobado.
- Tarjetas que siempre resultan en pago rechazado.
- Tarjetas que resultan en pago pendiente de revisión.

Estas tarjetas tienen números específicos documentados por MercadoPago y se utilizan con cualquier fecha de vencimiento futura y cualquier CVV. El entorno Sandbox es completamente independiente del entorno de producción y no procesa pagos reales.

11. Gestión de Estado con Zustand

Zustand es la librería elegida para gestionar el estado del cliente en el frontend de Food Store. A diferencia de soluciones más pesadas como Redux, Zustand ofrece una API minimalista sin boilerplate, excelente rendimiento gracias a suscripciones granulares, y un modelo mental simple basado en funciones.

Los Cuatro Stores

El estado del cliente se organiza en cuatro stores independientes, cada uno con una responsabilidad claramente delimitada:

****authStore**** gestiona todo lo relacionado con la sesión del usuario. Almacena el access token, el refresh token, los datos del usuario (nombre, email, roles), y el estado de autenticación (si está logueado o no). Expone acciones como ``login(tokens, user)``, ``logout()``, ``updateTokens(tokens)``, y selectores como ``isAuthenticated()``, ``hasRole(role)``. Este store se

persiste en `localStorage` mediante el middleware ``persist`` de Zustand, de modo que el usuario no pierde su sesión al cerrar el navegador. Se utiliza el parámetro ``partialize`` para excluir datos sensibles o transitorios de la persistencia si fuera necesario.

`**cartStore**` gestiona el carrito de compras. Almacena la lista de ítems (cada uno con el ID del producto, la cantidad, la personalización, y una copia del producto para mostrar en la UI). Expone acciones como ``addItem(producto, cantidad, personalizacion)``, ``removeItem(productoId)``, ``updateQuantity(productoId, cantidad)``, ``clearCart()``, y selectores como ``totalItems()``, ``totalPrice()``, ``getItem(productoId)``. Este store también se persiste en `localStorage`, permitiendo que el carrito sobreviva a cierres del navegador y a cambios de página. La persistencia del carrito es una de las funcionalidades más apreciadas por los usuarios — nada es más frustrante que perder un carrito armado con cuidado.

`**paymentStore**` gestiona el estado del proceso de pago. Almacena el estado actual del flujo de checkout (selección de dirección, selección de forma de pago, procesamiento, resultado), el ID de la preferencia de MercadoPago, el estado del pago, y mensajes de error si los hubiera. Expone acciones como ``startCheckout(pedidoId)``, ``setPreference(preferenciaId)``, ``updatePaymentStatus(status)``, y ``resetPayment()``. Este store NO se persiste en `localStorage` porque el estado del proceso de pago es inherentemente transitorio.

`**uiStore**` gestiona el estado de la interfaz de usuario. Almacena preferencias como el tema (claro/oscuro), el estado de sidebars y modals, notificaciones toast, y cualquier estado de UI que no pertenezca a otro store. Algunas partes de este store se persisten (como la preferencia de tema) y otras no (como el estado de un modal abierto).

Middleware de Persistencia

El middleware ``persist`` de Zustand serializa automáticamente el estado del store a `localStorage` cada vez que cambia, y lo rehidrata al iniciar la aplicación. Se configura con:

- Un nombre único para la clave en `localStorage` (por ejemplo, `"food-store-auth"`, `"food-store-cart"`).
- Una función ``partialize`` que define qué partes del estado se persisten. Por ejemplo, en el `authStore` se persisten los tokens y el usuario, pero no estados transitorios como `"isLoading"`.
- Opcionalmente, una función ``merge`` para controlar cómo se combinan el estado persistido con el estado inicial al rehidratar.

Buenas Prácticas Aplicadas

****Suscripción por slice****: En lugar de suscribirse al store completo (lo que causaría re-renders innecesarios cada vez que cualquier parte del store cambie), los componentes se suscriben a porciones específicas del estado. Por ejemplo, un componente que solo necesita mostrar la cantidad de ítems en el carrito se suscribe a ``useCartStore(state => state.items.length)`` en lugar de ``useCartStore()``. Esto garantiza que el componente solo se re-renderice cuando esa porción específica cambie.

****Acciones sin re-render****: Las funciones de acción del store (como ``addItem`` o ``login``) son estables y no cambian entre renders, por lo que se pueden extraer fuera del ciclo de renderizado. Esto evita que pasar acciones como props cause re-renders innecesarios en componentes hijos.

****getState fuera de React****: Zustand permite acceder al estado actual del store fuera de componentes React mediante ``useStore.getState()``. Esto es especialmente útil en los interceptores de Axios, donde se necesita leer el access token actual para adjuntarlo al header de cada petición, pero no se está dentro de un componente React donde se pueda usar el hook.

****Partialize para persistencia selectiva****: No todo el estado merece ser persistido. Los estados de carga, los errores transitorios, y los estados de UI efímeros no deben persistirse porque causan comportamientos inesperados al rehidratar (por ejemplo, un spinner de carga infinito si se persistió ``isLoading: true``).

12. Configuración y Setup

Variables de Entorno

Food Store requiere la configuración de diversas variables de entorno para funcionar correctamente. Estas variables se cargan desde un archivo ``.env`` que NUNCA se commitea al repositorio (está incluido en ``.gitignore``). En su lugar, se proporciona un archivo ``.env.example`` con las claves necesarias y valores de ejemplo.

Las variables del backend son:

****DATABASE_URL**** es la cadena de conexión a PostgreSQL en formato ``postgresql://usuario:password@host:puerto/nombre_db``. En desarrollo local, típicamente

apunta a ``postgresql://postgres:postgres@localhost:5432/foodstore``. En producción, se configura con las credenciales del servidor de base de datos desplegado.

****SECRET_KEY**** es la clave secreta utilizada para firmar los tokens JWT. Debe ser una cadena larga, aleatoria y criptográficamente segura. Se puede generar con ``openssl rand -hex 32``. Esta clave NUNCA debe exponerse públicamente — si se compromete, un atacante podría generar tokens válidos para cualquier usuario.

****JWT_ACCESS_TOKEN_EXPIRE_MINUTES**** define la duración del access token en minutos. El valor por defecto es 30. En desarrollo puede reducirse para testear el flujo de renovación. En producción, 30 minutos es un buen balance entre seguridad y experiencia de usuario.

****JWT_REFRESH_TOKEN_EXPIRE_DAYS**** define la duración del refresh token en días. El valor por defecto es 7, lo que permite a los usuarios mantener su sesión activa durante una semana sin necesidad de reingresar credenciales.

****CORS_ORIGINS**** es una lista de orígenes permitidos para peticiones cross-origin, separados por coma. En desarrollo incluye ``http://localhost:5173`` (el servidor de desarrollo de Vite). En producción, incluye el dominio de la aplicación frontend. Una configuración incorrecta de CORS es una de las causas más comunes de errores de comunicación entre frontend y backend.

****MERCADOPAGO_ACCESS_TOKEN**** es el token de acceso de la cuenta de MercadoPago. En desarrollo se usa el token del entorno Sandbox (que comienza con ``TEST-``). En producción se usa el token real (que comienza con ``APP_USR-``). Este token autoriza a Food Store a crear preferencias de pago y consultar estados de transacciones.

****MERCADOPAGO_PUBLIC_KEY**** es la clave pública de MercadoPago utilizada por el SDK de JavaScript en el frontend para la tokenización de tarjetas. A diferencia del access token, la clave pública está diseñada para ser expuesta en el código del cliente.

Las variables del frontend (prefijadas con ``VITE_`` según la convención de Vite) son:

****VITE_API_BASE_URL**** es la URL base del backend. En desarrollo es ``http://localhost:8000/api/v1``. En producción es la URL del servidor backend desplegado.

****VITE_MERCADOPAGO_PUBLIC_KEY**** es la clave pública de MercadoPago para el frontend, equivalente a la variable del backend pero accesible desde el código cliente.

Datos Semilla (Seed)

El sistema requiere datos iniciales que deben existir antes de que la aplicación pueda funcionar. Estos datos se cargan mediante un script de seed que se ejecuta después de las migraciones de base de datos.

Los **Roles** se cargan con cuatro registros: ADMIN (id: 1), STOCK (id: 2), PEDIDOS (id: 3) y CLIENT (id: 4). Estos IDs son estables y se referencian en el código.

Los **EstadoPedido** se cargan con seis registros: PENDIENTE (id: 1), CONFIRMADO (id: 2), EN_PREPARACIÓN (id: 3), EN_CAMINO (id: 4), ENTREGADO (id: 5) y CANCELADO (id: 6). Estos IDs también son estables y se usan en la lógica de la máquina de estados.

Las **FormaPago** se cargan con los métodos de pago aceptados: Tarjeta de crédito, Tarjeta de débito, y opcionalmente otros métodos según los requisitos del negocio.

El **usuario administrador** se crea con credenciales predefinidas (configurables por variables de entorno) y se le asigna el rol ADMIN. Este usuario es necesario para realizar la configuración inicial del sistema — crear categorías, cargar productos, y crear otros usuarios con roles específicos.

13. Patrones Aplicados

Food Store aplica un conjunto coherente de patrones de diseño que trabajan en conjunto para lograr un sistema mantenible, testeable y robusto. A continuación se describe cada patrón, su propósito y cómo se aplica en el sistema.

Repository

El patrón Repository abstrae el acceso a datos detrás de una interfaz de colección. En lugar de escribir queries SQL o llamadas a SQLAlchemy directamente en los servicios, cada entidad tiene un repositorio dedicado que expone métodos semánticos como ``get_by_id()``, ``list_all()``, ``create()``, ``update()`` y ``soft_delete()``. Esto permite que los servicios trabajen con objetos del

dominio sin preocuparse por los detalles de persistencia. Si en el futuro se necesitara cambiar de PostgreSQL a otro motor de base de datos, solo se modificarían los repositorios — los servicios permanecerían intactos. El `BaseRepository[T]` genérico evita la duplicación de código al proporcionar las operaciones CRUD comunes para cualquier entidad.

Unit of Work

Como se describió en detalle en la sección 9, el Unit of Work gestiona transacciones de base de datos agrupando múltiples operaciones en una unidad atómica. Su valor principal se manifiesta en operaciones complejas como la creación de pedidos, donde se necesita escribir en múltiples tablas de forma consistente. Sin el UoW, un error a mitad de la operación podría dejar la base de datos en un estado inconsistente (por ejemplo, un pedido creado sin detalles, o detalles creados sin pedido).

Service Layer

La capa de servicios concentra toda la lógica de negocio en un lugar predecible y testeable. Los routers delegan al servicio, y el servicio orquesta los repositorios a través del UoW. Esta separación tiene múltiples beneficios: la lógica de negocio se puede testear unitariamente sin necesidad de HTTP ni base de datos (inyectando mocks del UoW), los routers permanecen delgados y enfocados en HTTP, y la misma lógica de negocio puede ser invocada desde diferentes contextos (API REST, CLI, tareas asíncronas) sin duplicación.

Snapshot

El patrón Snapshot captura el estado de datos volátiles en un momento específico y lo almacena de forma inmutable. En Food Store se aplica en dos lugares críticos: los precios de productos en los detalles de pedido (`precio_snapshot`) y las direcciones de entrega en los pedidos (`direccion_snapshot`). Sin snapshots, si un administrador cambiara el precio de un producto, todos los pedidos históricos mostrarían el nuevo precio — claramente incorrecto. Los snapshots garantizan que cada pedido refleje fielmente las condiciones al momento de su creación, como una fotografía que no cambia aunque el sujeto se mueva.

Soft Delete

El borrado lógico (soft delete) marca registros como eliminados sin removerlos físicamente de la base de datos. Se implementa mediante el campo `eliminado_en` (timestamp nullable): si es null, el registro está activo; si tiene un valor, el registro fue "eliminado" en esa fecha. Todas las queries de lectura filtran automáticamente los registros eliminados. Este patrón ofrece

múltiples beneficios: recuperación de datos eliminados por error, preservación de integridad referencial (un producto eliminado que tiene pedidos asociados no rompe la base de datos), trazabilidad de quién eliminó qué y cuándo, y cumplimiento con regulaciones que requieren retención de datos.

Audit Trail (Append-Only)

El registro de auditoría de solo inserción se aplica en la tabla `HistorialEstadoPedido`. Cada transición de estado de un pedido genera un nuevo registro que documenta: el estado anterior, el nuevo estado, quién ejecutó la transición, cuándo, y una observación opcional. La regla cardinal es que esta tabla NUNCA se actualiza ni se elimina — solo se inserta. Esto proporciona una traza inmutable de la historia completa de cada pedido, invaluable para auditorías, resolución de disputas, y análisis operativo. Si un cliente reclama que su pedido estuvo "en camino" durante tres días, el historial tiene la evidencia exacta con timestamps.

Máquina de Estados Finitos (FSM)

La máquina de estados del pedido, descrita en detalle en la sección 5, es un patrón que modela las transiciones válidas entre estados de forma explícita y verificable. En lugar de permitir que cualquier código cambie el estado de un pedido arbitrariamente, la FSM define exactamente qué transiciones son válidas, desde qué estados, bajo qué condiciones, y por quién. Cualquier intento de transición inválida es rechazado con un error descriptivo. Esto previene estados inconsistentes (como un pedido que está "en camino" pero nunca fue "confirmado") y documenta las reglas de negocio de forma ejecutable.

Pagos Idempotentes

La idempotencia de pagos garantiza que una operación de pago se procese exactamente una vez, incluso si la solicitud se recibe múltiples veces. Se implementa mediante el `idempotency\_key` en la entidad `Pago`. Antes de procesar un webhook de MercadoPago, el sistema verifica si ya existe un registro con esa clave. Si existe, la notificación duplicada se ignora silenciosamente. Esto es crítico en sistemas de pago donde la duplicación puede significar cobrar dos veces al cliente — un error inaceptable que destruye la confianza.

Feature-Sliced Design

En el frontend, Feature-Sliced Design (FSD) organiza el código en capas horizontales (shared, entities, features, widgets, pages, app) con reglas estrictas de dependencia: cada capa solo puede importar de capas inferiores. Esto previene dependencias circulares, hace predecible

dónde encontrar y colocar código, y facilita el onboarding de nuevos desarrolladores. La estructura feature-first del backend es su contraparte del lado del servidor — en ambos casos, el principio es organizar por funcionalidad de negocio en lugar de por tipo técnico.

Custom Hooks

En el frontend de React, los custom hooks encapsulan lógica reutilizable que combina estado, efectos y queries. Por ejemplo, un hook `useProductos(filtros)` podría encapsular la query de TanStack Query para obtener productos, el manejo del estado de paginación, y la lógica de debounce para el filtro de búsqueda. Los componentes consumen estos hooks y se mantienen enfocados en la presentación visual, siguiendo el principio de separación de responsabilidades.

Optimistic Updates

Las actualizaciones optimistas son un patrón de UX donde el frontend actualiza la interfaz inmediatamente (de forma "optimista") antes de recibir la confirmación del servidor. Por ejemplo, cuando el usuario agrega un ítem al carrito, el carrito se actualiza visualmente al instante sin esperar una respuesta del servidor. Si la operación falla en el servidor, el cambio se revierte. TanStack Query proporciona soporte nativo para este patrón mediante las funciones `onMutate`, `onError` y `onSettled` de las mutations. El resultado es una experiencia de usuario que se siente instantánea incluso con latencia de red.

Webhook/IPN

El patrón de webhook permite que un servicio externo (MercadoPago) notifique a Food Store sobre eventos de forma asíncrona, sin necesidad de polling. En lugar de que Food Store consulte repetidamente a MercadoPago "¿ya se aprobó el pago?", MercadoPago envía un POST al webhook de Food Store cuando el estado del pago cambia. Este patrón es más eficiente (no hay requests innecesarios), más responsivo (la notificación llega en segundos), y más escalable (el servidor no gasta recursos en polling).

14. Rúbrica de Corrección

La evaluación del sistema Food Store se realiza sobre un total de **200 puntos** distribuidos en múltiples categorías que cubren tanto aspectos técnicos como funcionales del proyecto. La

rúbrica está diseñada para evaluar no solo si el sistema funciona, sino si está bien diseñado, correctamente implementado y adecuadamente documentado.

Categorías de Evaluación

****Modelo de datos y base de datos**** evalúa la correcta implementación del esquema de base de datos. Se verifica que todas las entidades del ERD v5 estén implementadas con los tipos de datos correctos, las restricciones de integridad referencial, los índices necesarios, la implementación de soft delete, los campos de auditoría, el uso correcto de arrays de PostgreSQL para personalización, y la estructura jerárquica de categorías con CTE. También se evalúa que las migraciones de Alembic estén correctamente definidas y sean reproducibles.

****Autenticación y autorización**** evalúa la implementación completa del sistema de seguridad. Se verifica el flujo JWT con access y refresh tokens, la rotación de refresh tokens, el hashing con bcrypt, la implementación de RBAC con los cuatro roles, la protección de endpoints con `require\_role`, el rate limiting en login, y la correcta configuración de CORS.

****API REST y lógica de negocio**** evalúa la calidad y completitud de la API. Se verifica que todos los endpoints estén implementados con los verbos HTTP correctos, que las respuestas sigan RFC 7807, que la paginación funcione correctamente, que las validaciones de Pydantic rechacen datos inválidos, que la máquina de estados del pedido se respete estrictamente con las cinco reglas de negocio, y que el patrón de snapshots esté correctamente aplicado.

****Unit of Work y Repository**** evalúa la implementación de los patrones de acceso a datos. Se verifica que el UoW gestione transacciones correctamente (commit en éxito, rollback en error), que el BaseRepository genérico esté correctamente tipado, que los repositorios especializados agreguen las queries necesarias, y que la creación de pedido sea atómica.

****Integración MercadoPago**** evalúa la correcta implementación del flujo de pagos. Se verifica la creación de preferencias con datos correctos, el procesamiento de webhooks IPN, la idempotencia de pagos, el manejo correcto de cada estado de pago, la tokenización PCI SAQ-A en el frontend, y el correcto uso del entorno Sandbox para pruebas.

****Frontend**** evalúa la implementación de la interfaz de usuario. Se verifica la arquitectura Feature-Sliced Design, la separación correcta de estado cliente (Zustand) vs estado servidor (TanStack Query), la implementación de los cuatro stores con persistencia, la configuración del interceptor de Axios, el uso de custom hooks, y la calidad visual y de UX de la interfaz.

****Patrones y arquitectura**** evalúa la correcta aplicación de los patrones descritos en la sección 13. Se verifica que cada patrón esté implementado correctamente y no solo mencionado en la documentación. Se penalizan las implementaciones que nombran un patrón pero no lo aplican realmente.

****Documentación y entrega**** evalúa la calidad de la documentación entregada, incluyendo el README con instrucciones de setup, la documentación de la API (generada automáticamente por FastAPI), el archivo `.env.example`, y el cumplimiento del checklist de entrega.

Escala de Calificación

La calificación final se calcula sobre los 200 puntos totales y se convierte a la escala que corresponda según los criterios institucionales. Se establece un umbral mínimo de aprobación que el estudiante debe superar para que el proyecto se considere aprobado.

Bonificaciones

Se otorgan bonificaciones adicionales para reconocer esfuerzo extra:

****+10 puntos por tests****: Si el proyecto incluye tests unitarios y/o de integración que cubran los flujos críticos del sistema. Se valora especialmente los tests de la máquina de estados del pedido, los tests del flujo de autenticación, y los tests de la creación de pedidos con su atomicidad.

****+10 puntos por deploy****: Si el proyecto está desplegado en un entorno accesible. Se acepta cualquier plataforma (Railway, Render, Fly.io, AWS, etc.) siempre que el sistema sea accesible y funcional. Se debe incluir la URL de acceso en la documentación.

Penalización

Se aplica una ****penalización del -30%**** sobre la nota final si el proyecto no cumple con los requisitos mínimos de entrega o si se detecta plagio. Los requisitos mínimos incluyen: que el sistema compile y se ejecute sin errores, que los endpoints principales estén funcionales, y que se hayan entregado todos los artefactos requeridos. El plagio se verifica tanto contra otros proyectos del curso como contra fuentes externas.

15. Entrega

El checklist de entrega define los artefactos que deben estar presentes y funcionales al momento de la entrega del proyecto. Cada ítem tiene un identificador para facilitar la referencia durante la corrección.

****CE-01 — Repositorio Git****: El proyecto debe estar versionado en un repositorio Git (GitHub, GitLab, o Bitbucket) con un historial de commits que demuestre trabajo progresivo. Repositorios con un solo commit masivo serán penalizados.

****CE-02 — README con instrucciones****: El repositorio debe contener un archivo README.md en la raíz con instrucciones claras para clonar, configurar y ejecutar el proyecto. Un evaluador debe poder levantar el sistema siguiendo exclusivamente las instrucciones del README.

****CE-03 — Archivo .env.example****: Debe existir un archivo `.env.example` que documente todas las variables de entorno necesarias con valores de ejemplo. NUNCA se debe incluir el archivo `.env` real en el repositorio.

****CE-04 — Migraciones de base de datos****: Las migraciones de Alembic deben estar incluidas en el repositorio y deben ser ejecutables de forma limpia en una base de datos vacía. El comando `alembic upgrade head` debe crear todas las tablas sin errores.

****CE-05 — Script de seed****: Debe existir un script o comando documentado para cargar los datos semilla (roles, estados de pedido, formas de pago, usuario admin). Sin estos datos, el sistema no puede funcionar.

****CE-06 — Backend funcional****: El backend FastAPI debe ejecutarse sin errores y responder a peticiones. La documentación automática de Swagger debe ser accesible en `/docs`.

****CE-07 — Frontend funcional****: El frontend React debe compilar sin errores y ser accesible desde el navegador. La navegación entre páginas debe funcionar correctamente.

****CE-08 — Flujo de autenticación completo****: Login, registro, renovación de tokens y logout deben funcionar de extremo a extremo.

****CE-09 — CRUD de productos****: La creación, lectura, actualización y soft delete de productos debe funcionar correctamente, incluyendo la gestión de categorías e ingredientes.

****CE-10 — Flujo de pedidos completo****: Debe ser posible crear un pedido, avanzar su estado a través de toda la máquina de estados, y verificar que el historial se registre correctamente. Las reglas de negocio (RN-01 a RN-05) deben respetarse estrictamente.

****CE-11 — Integración MercadoPago****: El flujo de pagos debe funcionar en el entorno Sandbox de MercadoPago. Se debe poder crear una preferencia, realizar un pago con tarjeta de prueba, y verificar que el webhook actualice correctamente el estado del pedido.

****CE-12 — Control de acceso por roles****: Cada rol debe tener acceso únicamente a los endpoints y funcionalidades que le corresponden. Un cliente no debe poder acceder a endpoints de administrador, y viceversa.

****CE-13 — Snapshots e inmutabilidad****: Los pedidos deben preservar los precios y direcciones al momento de su creación. Modificar un producto o dirección después de crear un pedido no debe alterar los datos del pedido existente.

****CE-14 — Carrito persistente****: El carrito de compras debe persistir en localStorage y sobrevivir al cierre del navegador, al refresh de la página, y al logout/login del usuario.

Este documento describe integralmente el sistema Food Store desde su concepción arquitectónica hasta los detalles de implementación y entrega. Cada sección está diseñada para proporcionar el contexto necesario para comprender, implementar y evaluar el sistema de forma completa. La clave del éxito en este proyecto no está solo en que el código funcione, sino en que cada decisión de diseño esté fundamentada, cada patrón esté correctamente aplicado, y cada flujo esté pensado de extremo a extremo.